Programming Assignment 3 Report

Question 1 - Arithmetic Coding With & Without E1/E2

Screenshots

- PA3_sample_inputs/q1_(1)_(2)_input_0.txt

```
String: 'ABAB'
Lower Bound (without E1/E2): 0.3125
Upper Bound (without E1/E2): 0.375
Interval (without E1/E2): [0.3125, 0.375)
E1/E2 Operations: ['E1', 'E2']
```

- PA3_sample_inputs/q1_(1)_(2)_input_1.txt

```
String: 'AABBABBA'
Lower Bound (without E1/E2): 0.2109375
Upper Bound (without E1/E2): 0.21484375
Interval (without E1/E2): [0.2109375, 0.21484375)
E1/E2 Operations: ['E1', 'E1', 'E2', 'E2', 'E1', 'E2', 'E2']
```

- PA3_sample_inputs/q1_(1)_(2)_input_2.txt

```
String: 'AAAAAB'
Lower Bound (without E1/E2): 0.3348979766803842
Upper Bound (without E1/E2): 0.401877572016461
Interval (without E1/E2): [0.3348979766803842, 0.401877572016461)
E1/E2 Operations: ['E1', 'E2']
```

- PA3_sample_inputs/q1_(1)_(2)_input_3.txt

```
String: 'BBBBBA'
Lower Bound (without E1/E2): 0.5981224279835391
Upper Bound (without E1/E2): 0.6651020233196159
Interval (without E1/E2): [0.5981224279835391, 0.6651020233196159)
E1/E2 Operations: ['E2', 'E1']
```

Challenges While Implementing Arithmetic Coding:

- Ensuring the correct handling of probabilities and updating bounds accurately with each symbol encountered.
- Identifying correct order to apply E1/E2 operations while maintaining interval.
- Handling corner cases—empty strings or strings with only one type of character.
- Dealing with precision of floating point numbers. As coding process narrows interval, difference between high and low bound becomes very small (especially in long sequences), requiring higher precision to maintain accuracy where precision loss could accumulate.
- Arithmetic coding is a complex algorithm, so my understanding had to be concrete in order to prevent an error prone implementation, especially scaling and normalization of intervals with E1/E2 operations.

Steps to Run the Code

1. In terminal, please cd to the assignment directory, and type ‘python q1.py’ or ‘python3 q1.py’ (Linux/MacOS).
2. Given strings and their bounds, intervals, E1/E2 operations will be output.

Question 2 - Discrete Cosine Transform

Screenshots

- **Section 1 (Input 0)** - PA3_sample_inputs/q2_(1)_input_0.txt

```
PA3_sample_inputs/q2_(1)_input_0.txt: Size = 1

Transformation Matrix:
[[1.]]
```

- **Section 1 (Input 1)** - PA3_sample_inputs/q2_(1)_input_1.txt

```
PA3_sample_inputs/q2_(1)_input_1.txt: Size = 4

Transformation Matrix:
[[ 0.5         0.5         0.5         0.5       ]
 [ 0.65328148  0.27059805 -0.27059805 -0.65328148]
 [ 0.5        -0.5        -0.5         0.5       ]
 [ 0.27059805 -0.65328148  0.65328148 -0.27059805]]
```

- **Section 1 (Input 2)** - PA3_sample_inputs/q2_(1)_input_2.txt

```
PA3_sample_inputs/q2_(1)_input_2.txt: Size = 5

Transformation Matrix:
[[ 4.47213595e-01  4.47213595e-01  4.47213595e-01  4.47213595e-01
   4.47213595e-01]
 [ 6.01500955e-01  3.71748034e-01  3.87267321e-17 -3.71748034e-01
  -6.01500955e-01]
 [ 5.11667274e-01 -1.95439508e-01 -6.32455532e-01 -1.95439508e-01
   5.11667274e-01]
 [ 3.71748034e-01 -6.01500955e-01 -1.16180196e-16  6.01500955e-01
  -3.71748034e-01]
 [ 1.95439508e-01 -5.11667274e-01  6.32455532e-01 -5.11667274e-01
   1.95439508e-01]]
```

- **Section 2 (Input 0)** - PA3_sample_inputs/q2_(2)_input_0.txt

```
PA3_sample_inputs/q2_(2)_input_0.txt
Original Vector = [0. 1. 2. 3. 4.]

Transformed Vector:
[ 4.47213595 -3.14949989 -0.         -0.28399023 -0.        ]
```

- **Section 2 (Input 1)** - PA3_sample_inputs/q2_(2)_input_1.txt

```
PA3_sample_inputs/q2_(2)_input_1.txt
Original Vector = [ 100.  200. -100.    0.    1.    0.]

Transformed Vector:
[  82.05790638  122.06623885  100.           0.40824829 -116.04740411
 -122.06623885]
```

- **Section 2 (Input 2)** - PA3_sample_inputs/q2_(2)_input_2.txt

```
PA3_sample_inputs/q2_(2)_input_2.txt
Original Vector = [0. 0. 0. 0. 0. 0.]

Transformed Vector:
[0. 0. 0. 0. 0. 0.]
```

- **Section 3**

```
High frequency Vector: [ 120.0 -200.0 100.0 30.0 78.0 10.0 8.0 5.0 ]

Transformed Vector:
[ 53.38656  -9.75919 -49.93222  37.29724 111.36932 122.77075 142.75862
 129.95198]
```

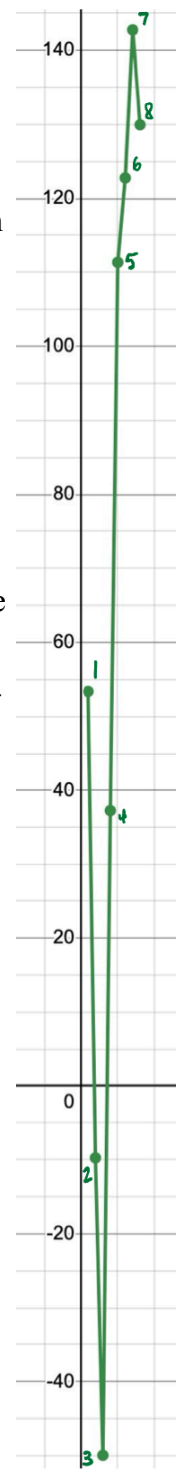
Values (especially small, such as ^-16) could be subject to float precision errors since integers multiplied/added w/ small values (0.xxx, so on)

Regan Li
301426946
CMPT 365 - E 100

Section 3 - Designing Input Vector

Included in "`PA3_sample_inputs/q3.txt`" is vector [120, −200, 100, 30, 78, 10, 8, 5], and its Discrete Cosine Transform [53.38656 -9.75919 -49.93222 37.29724 111.36932 122.77075 142.75862 129.95198] represents a vector with high energy in higher frequency components, while being low energy in low frequency components. The DCT result (shown in plot to right) shows significant energy in higher frequencies (components towards end of the vector), which indicates signals with rapid change/variation.

An example of a signal with high energy in higher frequency components and low energy in lower frequency components could be the sound of a chirping bird in an environment with a quiet background. A chirping sound represents such a vector because the quiet background represents low energy in lower frequency components, as it lacks significant variations or changes over time. Meanwhile, the chirping sound contains rapid fluctuations in amplitude and frequency, representing high energy in the higher frequency components. Chirps are brief, high pitched, and vary quickly in amplitude, fitting characteristics of said input vector.



(Created Using Desmos)

Steps to Run the Code

1. In terminal, please cd to the assignment directory, and type 'python q2.py' or 'python3 q2.py' (Linux/MacOS).
2. Given N values and vectors, corresponding results will be output (incl. S3).